CSCI 377: Computer Algorithms

John Jay College of Criminal Justice — CUNY

Project Report

Final Documentation

Project Title:	Library Management System
Team Members:	Ethan Winch, ethan.winch@jjay.cuny.edu
	Shalin Valerio, shalin. shalin.valerio@jjay.cuny.edu
GitHub Repository:	https://github.com/ethwinch/Library-Management-System
Date Submitted:	May 11th, 2026

Due Date: 11 May 2026

Table of Contents

Table of Contents

Abstract

Write a concise summary (150–250 words) of your entire project here.

> **Instructor Note:** *Summarize: (1) the problem you addressed, (2) the algorithm(s) you chose, (3) the main data structures used, (4) your key experimental finding — did real-world timing match the theoretical complexity? (5) the most important conclusion.*
>
> *Write this LAST, after all other sections are complete.*
>
> *No citations, diagrams, or code in the abstract — prose only.*

This Library Book Management System allows users to borrow, return, and browse books within the book database. The system keeps track of members and books, meaning that if a book is unavailable while a member tries to borrow it, they will be added to the waitlist for that book.

The algorithms linear and binary are used for searching and sorting. The vector data structure is used for keeping track of the book catalogue and member waitlist for individual books. Classes are used to structure member and book data.

While experimenting, we found binary search and sort to be the fastest while linear search and sort was slow. Binary search was significantly faster than linear search, which is in line with the time complexity of the algorithms, with binary search having a worst case of O(1) and a best case of O(log n), and linear search being O(n) and O(1) respectively. Quick sort had a time complexity of O(n log n) on average, but $O(n^2)$ in the worst case. Merge sort had a complexity of O(n log n), but performed significantly worse in practice due to the copying of vectors. As expected, binary search performed the best out of all the algorithms. As more data was introduced, the runtimes matched accordingly for all algorithms (except merge sort, which performed significantly worse as data increased).

In conclusion, binary search was the quickest algorithm tested, allowing for ISBNs to be found in O (log n) time, even on large datasets.

1. Introduction

1.1 Problem Statement

Describe the real-world problem your system addresses.

> **Instructor Note:** *Explain the problem in plain English — as if speaking to a non-programmer. What task needs to be automated or optimized? Why does it matter? Reference the specific topic you chose (e.g., Banking Management, City Bike Planner, etc.).*
>
> *Aim for 3–5 sentences.*

For a Library Management System, it is expected that users can borrow, return, and browse books. If users try to borrow a book that is currently unavailable, they will be added to the waitlist for that book. Automating these tasks allow for more efficient handling of the book catalogue as well as keeping track of who has borrowed what books. Automating these processes help prevent human error and allow for easy communication across libraries.

1.2 Project Objectives

List the specific goals your project aims to achieve.

> **Instructor Note:** *Each objective should be specific and measurable. Examples: 'Implement Dijkstra's algorithm to find shortest bike routes' or 'Demonstrate that sorting accounts by balance runs in O(n log n) time and comparing it to another algorithm that runs in O(n) and $O(n^2)$ experimentally.'*
>
> *List 3–5 objectives as bullet points.*

- Implement Binary Search & Sort for the quickest ISBN lookup.
- Compare Binary Search/Sort to Linear Search/Sort to demonstrate the drastic time differences between O(n) and O(log n) time complexities on large data sets.
- Allow users to borrow, return, and search for books by ISBN.
- Track members and what books they have checked out.

2. System Architecture

2.1 Data Structures Used

Identify and justify every major data structure your system uses.

Instructor Note: *For EACH data structure (e.g., array, linked list, hash map, graph, BST, heap), answer: (a) What is it used for in your system? (b) Why was it the right choice? (c) What is the time complexity for the main operations you perform on it?*

Use the table below — add rows as needed.

Data Structure	Role in System	Justification & Key Complexity
Vector	*Stores books in catalogue and member waitlist for checking out currently unavailable books.*	*Simple, efficient and widely supported C++ library. O(1) time complexity for insertion.*
Member Class	*Structure for the member object to store member data.*	*Organizes and structures data while enabling waitlists and tracking the holders of current books.*
Book Class	*Structure for the book object to store book data.*	*Organizes and structures data for easy access and keeping data organized.*

2.2 Module / Component Description

Briefly describe each module or major function group.

Instructor Note: *Think of modules as logical groupings of functions e.g., 'Input/Output Module', 'Sorting Module', 'Search Module', 'Graph/Routing Module'. For each module, state its responsibility and the key algorithms or operations it contains.*

Do NOT paste code here. Describe behavior and purpose only.

Books:

Checkout Function: If the book is available, have the member check it out. If it's unavailable, add the member to the book's waitlist.

Return Function: Remove the book from the member's "borrowed books" vector. If the book has someone in its waitlist, automatically have the next person in the waitlist checkout the book.

Linear Search: Search each index chronologically of the catalogue vector for the specified ISBN.

Binary Search: Search for the given ISBN by halving the catalogue every iteration. Start from the middle of the vector, then compare the target value to the value at the current index. If the current value is less than the target, take the left half of the vector; else take the right half. Then find the middle of the selected half and repeat until the value at the current index matches the target.

Quick Sort: Sort the books by selecting a pivot element from the vector, then partitioning all other elements into two groups which would be less than or greater than the pivot's ISBN. This is applied recursively to both groups until it's properly sorted.

Merge Sort: Sort the catalogue by repeatedly dividing the vector into smaller halves until each vector contains one element. Then merge the vectors back by comparing the associated ISBN for each half and sorting appropriately.

3. Algorithm Design

3.1 Algorithm Selection & Rationale

Identify every core algorithm your project uses and justify your choices.

> **Instructor Note:** *For each algorithm (e.g., Merge Sort, Dijkstra's, Binary Search, Greedy cash-flow minimizer), explain: (a) What problem does it solve in your system? (b) Why did you choose THIS algorithm over alternatives? (c) What are the trade-offs (time vs. space, simplicity vs. optimality)?*
>
> *Refer to the project topic e.g., if you chose the City Bike Planner, explain why Dijkstra's is appropriate vs. Bellman-Ford.*

Binary Search/Sort: Fast ISBN lookup, even on large datasets. It has a more complex implementation, but allows for fast lookup times that become more apparent when searching/sorting large data sets.

Linear Search/Sort: For ISBN lookup. It works fine on small datasets, but the seconds add up on data sets with more data to search through. Trades speed for simplicity.

Merge Sort: Attributed as a good stable choice for large datasets. It was chosen as it was considered O * log n in all cases but at the cost of a costly copying process depending on the data that is manipulated.

Quick Sort: Also considered a great but less stable option for large datasets. Due to the potential possibility of a bad pivot it can lead to a larger worse case situation but it does have less copy actions initiated.

3.2 Pseudocode

Write clear pseudocode for each core algorithm.

> **Instructor Note:** *Pseudocode should be language-agnostic — no Python, Java, or C++ syntax. Use plain English + structured notation (e.g., FOR, WHILE, IF/ELSE, RETURN). Number the lines.*
>
> *One pseudocode block per algorithm. Each block should start with the algorithm name and its inputs/outputs.*
>
> *Do NOT paste your actual source code here.*

```
Algorithm: Binary Search   |   Input: Book vector & target ISBN   |
Output: "ISBN found at x index" or "ISBN not found"
1.  low = zero
1.  high = length of book vector - 1
1.  found = false
2.  WHILE low <= high:
3.      IF ISBN at current book index equals the target ISBN, THEN
4.          Print "target found at x index"
3.      IF target ISBN > ISBN at current book index THEN
4.          low = middle index + 1
3.      ELSE
4.          IF middle is zero -> break
4.          Set high to the current value of middle
5.  IF found is not true
5.      print "ISBN not found"
```

```
Algorithm: Linear Search   |   Input: Book vector & target ISBN   |
Output: "ISBN found at x index" or "ISBN not found"
1.  counter = 0
2.  WHILE counter < Book vector size and ISBN of book at the index counter
does not equal target:
3.      increment counter
6.  IF counter equals Book vector size:
6.      print "ISBN found"
7.  ELSE
8.      print "ISBN not found"
```

```
Algorithm: Merge Sort   |   Input: Book vector, left, mid, right   |
Output: merged vector
1.  // MERGE
1.  merge(vector books, left, mid, right)
1.    n1 = mid - left + 1
1.    n2 = right - mid
1.    Reserve n1 space in vector L
1.    Reserve n2 space is vector R
2.    FOR each element in L up to index n1:
3.        Insert left half of books into L vector
2.    FOR each element in R up to index n2:
3.        Insert right half of books into R vector
3.    i = 0
3.    j = 0
3.    k = left
3.    WHILE i < n1 and j < n2:
5.        IF left ISBN <= right ISBN:
5.           books[k] = left ISBN
```

```
5.           increment i
5.        ELSE
5.           books[k] = right ISBN
5.           increment j
5.        increment k
6.    WHILE i < n1:
6.        books[k] = left ISBN
6.        increment i and k
6.    WHILE j < n2:
6.        books[k] = right ISBN
6.        increment j and k

7. // MERGE SORT
7.  mergeSort(book vector, left, right)
7.    IF left >= right:
7.      Return
7.    mid = left + (right - left) / 2 // Calculate new mid based on right
                                          and left values
7.    RECURSE passing book vector, left and mid
7.    RECURSE passing book vector, mid + 1 and right
8.    merge(books, left, mid, right)
```

```
Algorithm: Quick Sort   |   Input: Book vector, low, high   |   Output:
partition outputs pivot, quicksort outputs a sorted vector
1.  partition(Books vector, low, high):
1.    pivot = ISBN of book at high index
1.    i = low - 1
2.  FOR each element from low to high:
3.      IF current ISBN <= pivot THEN
4.          increment i and swap book at i with book at j
5.  swap books at i=1 and high
6.  RETURN i+1
6.
6.  quickSort(book vector, low, high)
6.      IF low < high THEN:
6.        pi = partition(books, low, high)
6.        RECURSE passing books vector, low, pi - 1
6.        RECURSE passing books vector, pi+1, high
```

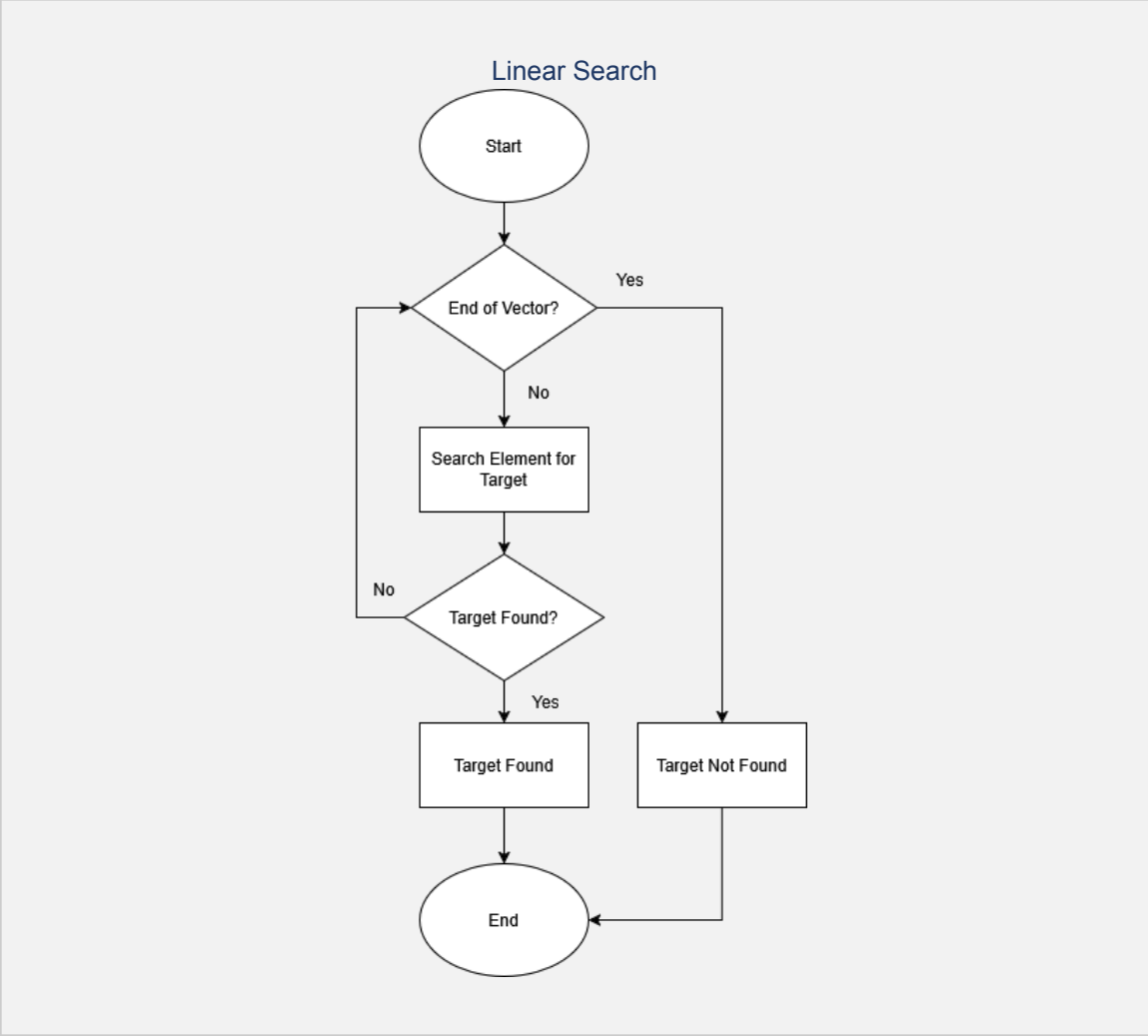
3.3 Flowchart / Process Diagram

Include a visual flowchart of the algorithm's main logic or system workflow.

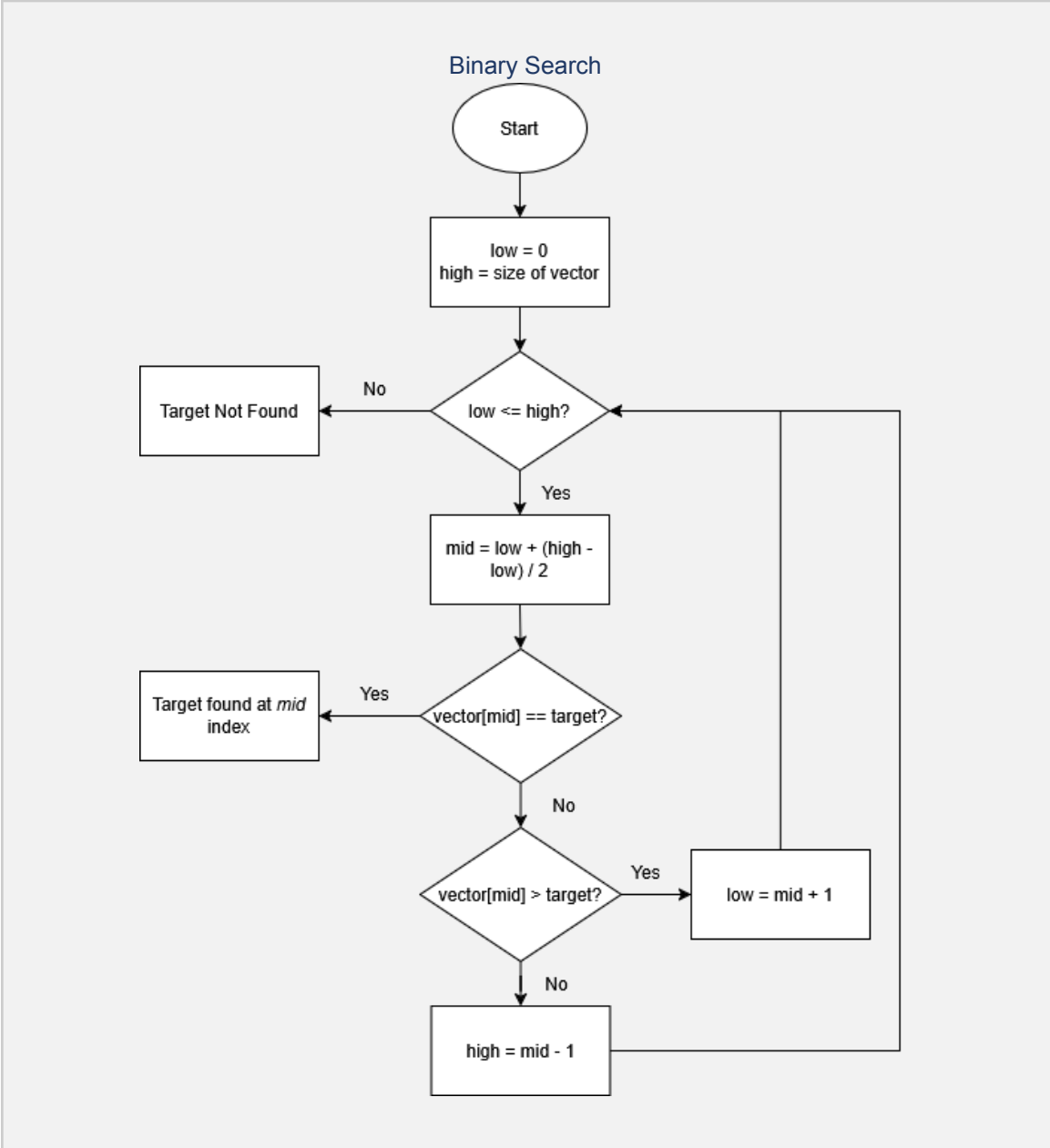
Instructor Note: *Draw a flowchart that captures the key decision points and loops in your algorithm. Use standard symbols: ovals for start/end, rectangles for processes, diamonds for decisions.*

Tools: draw.io (free), Lucidchart. Paste the image below.

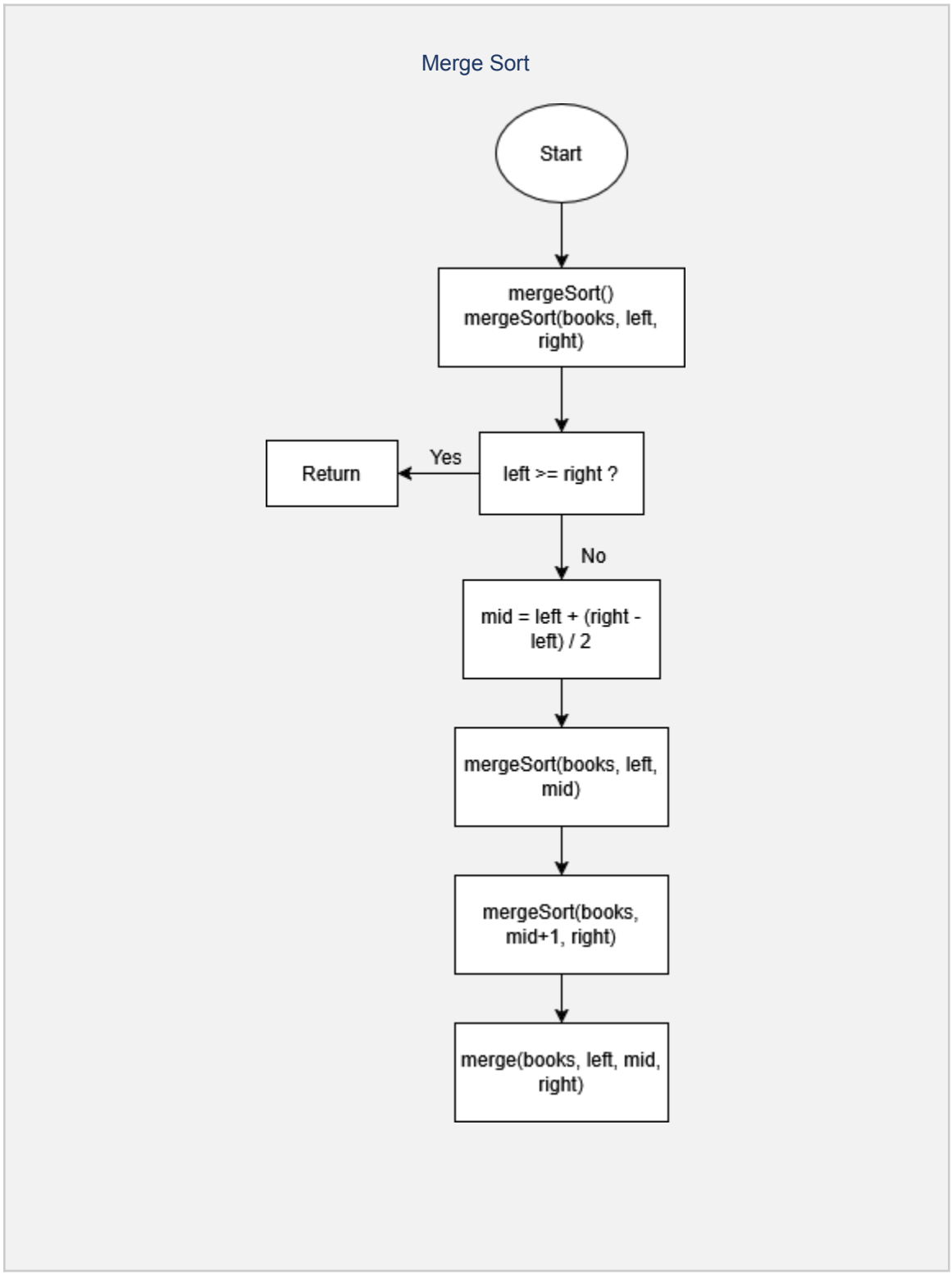
Beneath the diagram, write 2–3 sentences explaining the flow.

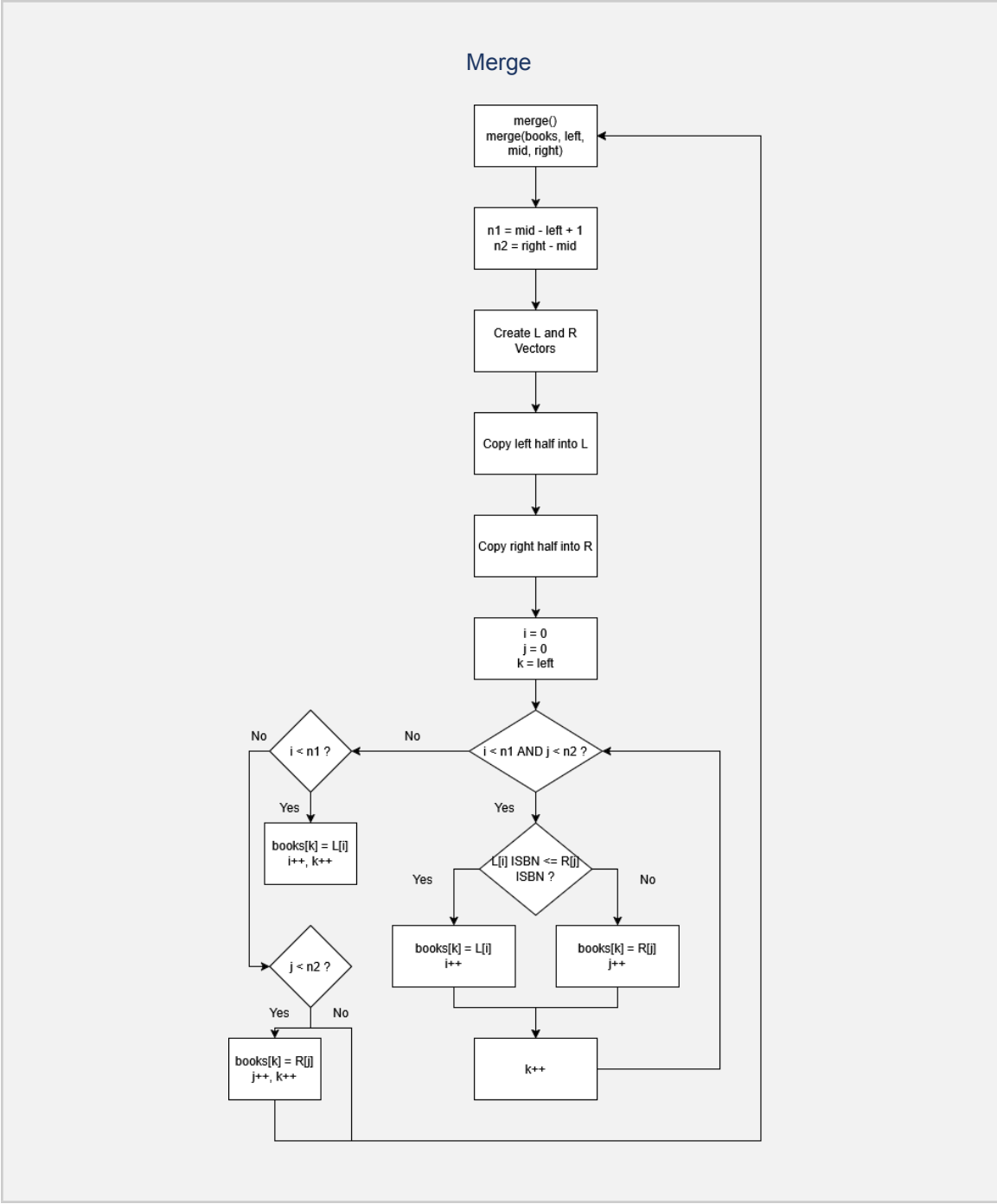


Check that it's not the end of the vector. If it's the end then return that the target was not found, otherwise check if the element matches the target. If it matches, return that the target has been found and terminate the algorithm, otherwise repeat until the end of the vector is reached.

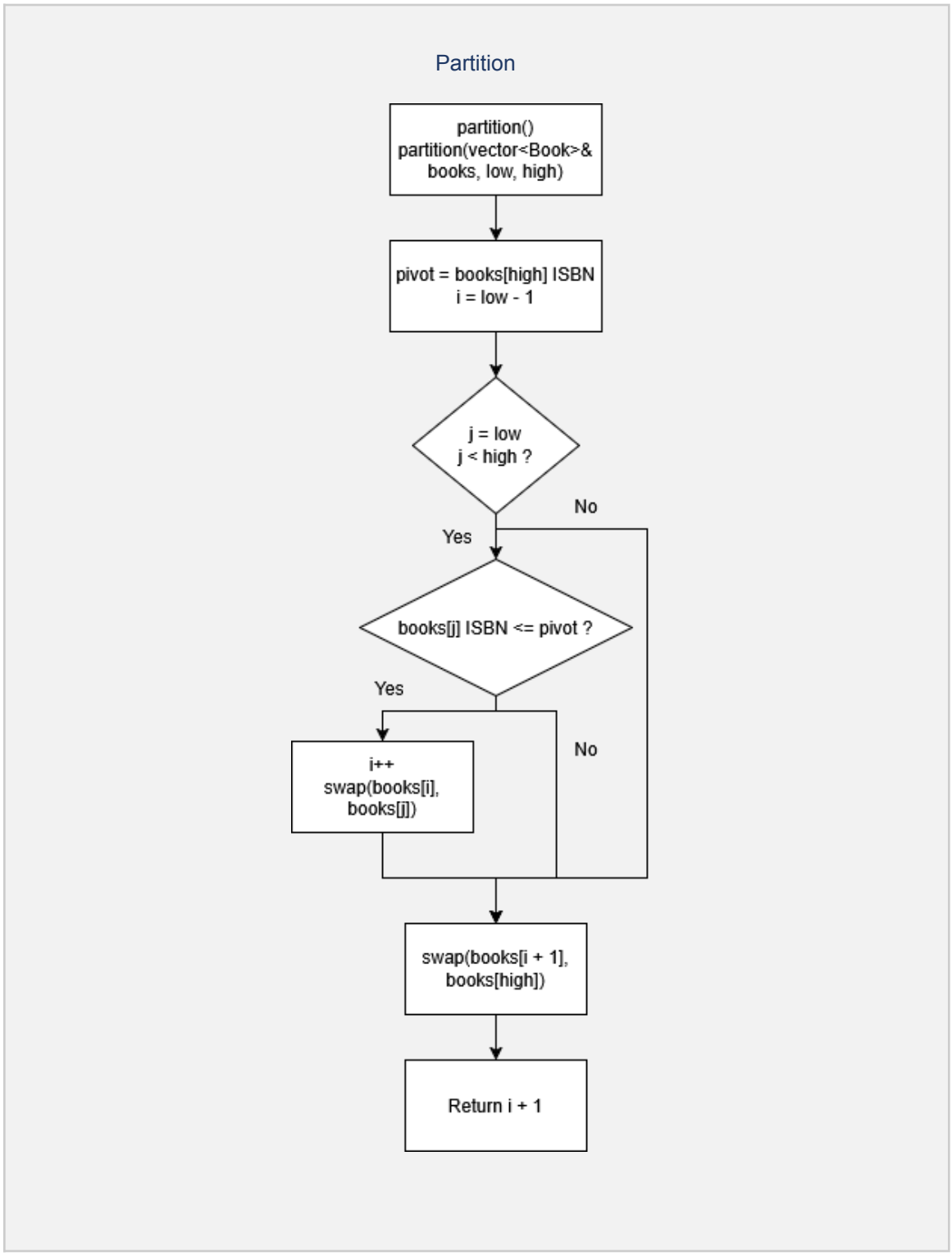


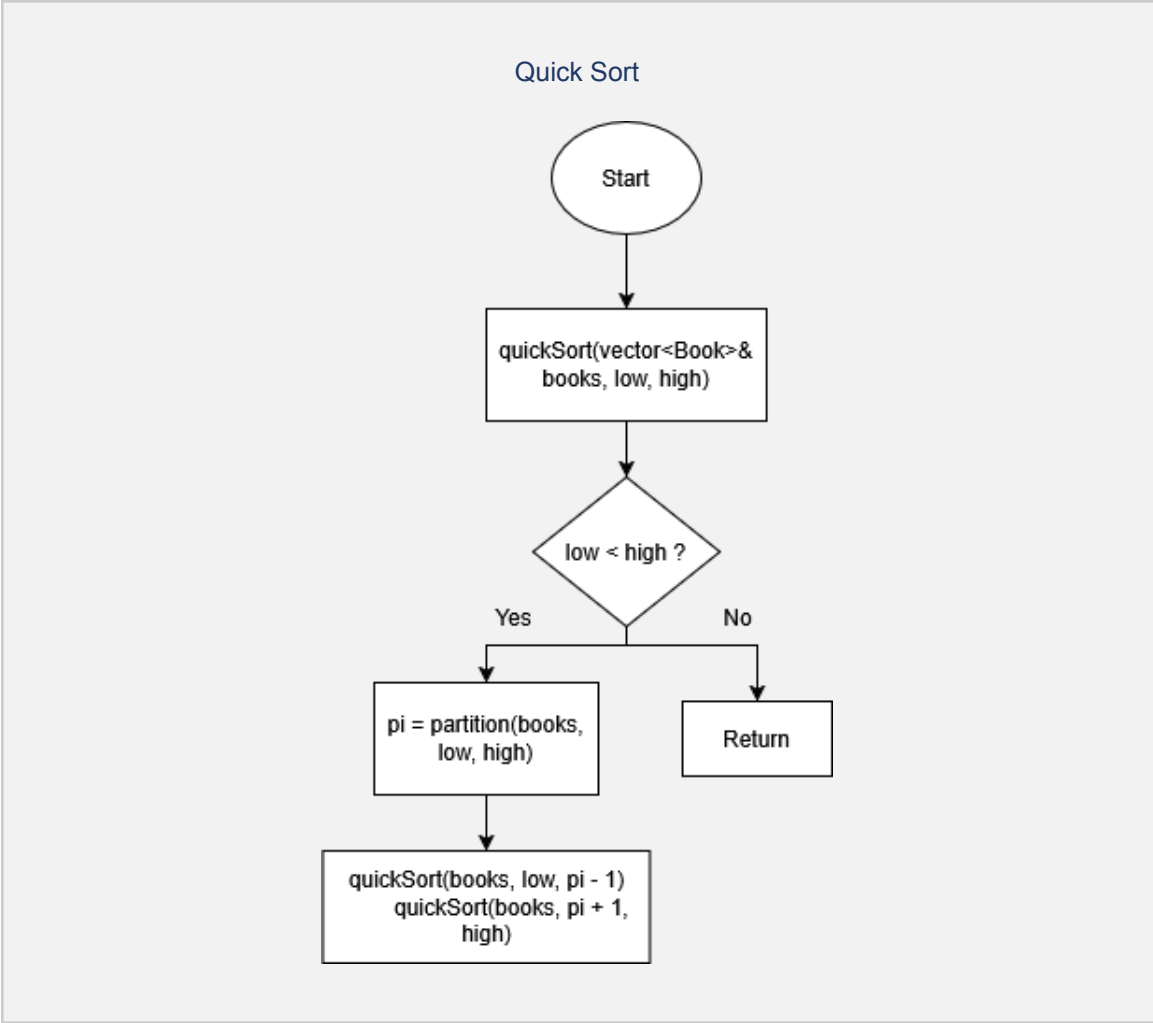
While low is less than high, take half the sum of low and high to get the middle of the vector. If the value at the middle of the array is equal to the target, return that the target was found at index mid. Otherwise, if the value is less than the target, set the value of high to one less than the current mid index. If the value is greater than the target, set the low to one more than the current mid index. Continue this until the target is found or low is less than or equal to high, in which case the target value is not found in the vector.


Merge Sort
Start
mergeSort()
mergeSort(books, left, right)
left >= right ?
Yes
Return
No
mid = left + (right - left) / 2
mergeSort(books, left, mid)
mergeSort(books, mid+1, right)
merge(books, left, mid, right)



Calculate the middle of the vector, then copy the left half of the vector into vector L, and right half into vector R. For subvectors R and L, divide them in half based on the mid-point until each subvector only consists of one element. Then, combine the arrays using merge, which compares each element in the vector and combines them in a new vector in a sorted order.


Partition
partition()
partition(vector<Book>&
books, low, high)
pivot = books[high] ISBN
i = low - 1
j = low
j < high ?
No
Yes
books[j] ISBN <= pivot ?
Yes
No
i++
swap(books[i],
books[j])
swap(books[i + 1],
books[high])
Return i + 1



In partition(), a pivot is chosen so the vector is rearranged so that all elements less than the pivot are placed to the left of the pivot and all elements greater than the pivot are placed to the right of the pivot. The pivot is returned to quickSort() which implements divide and conquer. It divides the vector into two subvectors based on the returned pivot and recursively calls itself for the two subvectors until the whole vector is sorted.

4. Time Complexity Analysis

Provide a formal Big-O analysis of each algorithm.

Instructor Note: *For EACH algorithm, derive its time. Show your reasoning, not just the final answer. For example, explain why a nested loop over n elements gives O(n²), or why each level of a divide-and-conquer recursion contributes O(n) work leading to O(n log n) overall.*

Clearly state the Best Case, Average Case, and Worst Case for each algorithm.

Algorithm	Best Case	Average Case	Worst Case
Binary Search	*O(1)*	*O(log n)*	*O(log n)*
Linear Search	*O(1)*	*O(n)*	*O(n)*
Merge Sort	*O(n log n)*	*O(n log n)*	*O(n log n)*
Quick Sort	*O(n log n)*	*O(n log n)*	*O(n²)*

BINARY SEARCH

The best case is that the target is at the middle of the vector. Otherwise, the complexity is log n because it repeatedly cuts the elements to be searched in half.

LINEAR SEARCH

The best case is that the target is the first element of the vector. Otherwise, the complexity is the length of the vector since it searches each element one by one from beginning to end. The worst case is if the target is the last element in the vector.

MERGE SORT

All cases are the same time complexity because the algorithm recursively divides the vector in half, then sorts the two halves before merging them back together.

QUICK SORT

The best and average case is the same as merge sort with its divide and conquer approach with the addition of choosing a pivot at a random point in the vector. The worst case, however, occurs when the last element is chosen as the pivot in an already sorted vector.

5. Experimental Setup and Results

5.1 Timing Methodology

Explain precisely how you measured the runtime of your algorithms.

Instructor Note: *Describe the timing mechanism used in your programming language:*

Python: time.perf_counter() or time it module

Java: System.nanoTime()

C/C++: clock() from <time.h> or chrono library

Also explain: (a) What exactly did you time — just the algorithm call, or the full function including I/O? (b) How many times did you repeat each measurement to get a stable average? (c) Did you warm up the system (e.g., discard the first run)? (d) What machine did you run tests on (rough specs — CPU, RAM)?

For timing, we used the chrono library. To determine the runtime of each algorithm, we started the timer at the start of the function. We ran the function five times for each algorithm with each iteration using more data (from 100 to 1,000,000 books). The five iterations were then averaged to give a more stable measurement. There was no need to warm up the system. Although possible outliers could exist, it's unlikely for them to occur within three separate runs. The algorithms were run on a machine with an 11th gen intel core i5-1145G7 2.60 GHz processor and 16384 MB of RAM.

5.2 Results Table

Record measured runtimes for each input size and algorithm.

Instructor Note: *Fill in the measured average runtime (in milliseconds or microseconds — be consistent). If you tested multiple algorithms, create a separate table per algorithm OR add more columns.*

Report the average of at least 3 runs per configuration. Include the number of runs averaged.

BINARY SEARCH

Input Size (n)	Run 1 (µs)	Run 2 (µs)	Run 3 (µs)	Average (µs)
100	1.724	1.756	1.229	1.56967
1000	3.495	2.286	2.240	2.67367
10000	4.583	1.855	2.422	2.95667

Input Size (n)	Run 1 (µs)	Run 2 (µs)	Run 3 (µs)	Average (µs)
100000	7.182	2.448	6.414	5.68133
1000000	6.833	4.864	4.495	5.39733

LINEAR SEARCH

Input Size (n)	Run 1 (µs)	Run 2 (µs)	Run 3 (µs)	Average (µs)
100	0.877	0.875	0.650	0.800667
1000	8.421	4.482	5.675	6.19267
10000	85.430	51.478	33.699	56.869
100000	816.465	344.877	135.213	432.185
1000000	2892.830	1857.477	968.213	1906.17

QUICK SORT

Input Size (n)	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)
100	0.413699	0.342765	0.202489	0.319651
1000	3.409856	4.042576	3.686791	3.71307
10000	49.460397	57.513978	50.855801	52.6101
100000	627.124989	584.600552	742.253254	651.326
1000000	15892.175217	16273.966728	14894.873706	15687

MERGE SORT

Input Size (n)	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)
100	0.600213	0.302192	0.300982	0.401129
1000	4.341024	4.522307	6.207194	5.02351
10000	67.804307	70.180904	68.389918	68.7917

Input Size (n)	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)
100000	1166.56992	1211.83079	1173.936420	1184.11
1000000	29032.238504	31166.74152	29514.464493	29904.5

5.3 Interpretation: Theory vs. Practice

Analyze whether your experimental results match theoretical predictions.

Instructor Note: *This is the CORE analytical section — worth 3 points in the rubric. Answer all of the following:*

(a) Did your measured runtime grow as predicted by your Big-O analysis? (e.g., doubling n — did time roughly double for O(n), or quadruple for O(n²)?)

(b) Were there any surprises or anomalies? (e.g., an algorithm was faster than expected at small n, or slower due to memory overhead)

(c) What real-world factors might explain any gap between theory and practice? (e.g., caching effects, constant factors, language overhead, hardware speed)

(d) Did one algorithm significantly outperform another? What does this mean for your system?

Write at least 2–3 paragraphs.

As predicted, the growth rate matches the Big-O analysis. For linear search, as the input size increased, the search time doubled. For binary search, the time increased by only a few nanoseconds each iteration, even as the input size increased dramatically, which is inline with O(log n) time complexity. Quicksort and mergesort both had longer run times with more data. Quicksort typically performed better than its worst case of O(n²), but still became significantly slower with more data, as expected due to its best and average O(n log n) time complexity.

Merge sort performed worse than quicksort despite supposedly having a faster worst case of O(n log n). Quick sort only copies a vector once for the purposes of having a vector to the left and row the right then follows that recursively while merge sort does the same break down but on both sides (right and left) causing more vectors to be copied which is more costly than quick sort.

In real-life applications, datasets are often massive which highlights any compounding inefficiencies in sorting/searching algorithms. Binary search out-performed the others by far on larger sets of data while the other methods lagged behind as the data increased. This was expected given the best, worst, and average cases for their respective time complexities. Binary search and sort should be used as it was able to maintain a low search/sort time no matter the amount of data it needed to search/sort.

6. Design Decisions and Discussion

6.1 Challenges Encountered

Describe the most significant technical or logistical obstacles your team faced.

> **Instructor Note:** *Be specific and honest. Describe: What was the challenge? When did you encounter it? How did you (or did you not) resolve it?*
>
> *Examples: debugging an off-by-one error in graph traversal, performance bottleneck in a nested loop, difficulty generating fair test data, coordinating work between team members.*

Loading the Dataset: Figuring out how we wanted to load the dataset was the most time consuming. We initially loaded the data into an SQL database, but since SQL already has built in functions for sorting and searching, we had to pivot to loading the dataset directly into C++ so we could write our own algorithms.

Book and Member Classes: The Book and Member classes reference one another, but initially there was an error where C++ was not allowing the classes to reference one another, despite being friend classes. In the end, we had to add “class Member;” to the top of the Book header file and “class Book;” to the top of the Member header file so each class could read from the other.

6.3 Limitations

Acknowledge what your current implementation does NOT do well.

> **Instructor Note:** *Every system has limitations. Acknowledging them shows maturity and critical thinking. Consider:*
>
> *- Scale: Does your system break or slow dramatically at very large inputs?*
>
> *- Correctness: Are there edge cases your system does not handle?*
>
> *- Real-world applicability: What would need to change for this to be a production system?*

- Modularity was a fairly neglected aspect of the .h files. We would have liked to follow the traditional aspect of a function does one thing throughout all .h files
- Due to our usage of a vector of classes the merge and quick sorts are completely bogged down due to how costly the action of copying over a vector is, especially one that contains string values. This does not create a level playing field for comparison as quicksort creates fewer copy actions leading to a favorable result strictly due to the type of data it manages.

- Nearly every aspect of our system assumes that the vectors will be filled with data and the ISBN's are within the system. There are no appropriate catch and throw cases that cater to an empty csv file or an ISBN that is not on the list.

7. Reflection and Lessons Learned

7.1 What Worked Well

Instructor Note: *Reflect on the successes of your project. What aspects of your design, implementation, or teamwork went smoothly? What are you most proud of?*

Be specific — not just 'everything worked' but e.g., 'Our graph construction correctly modeled the city network and Dijkstra's produced accurate shortest paths on all test cases.'

Using classes and vectors to structure the data made it easier when creating the sorting and searching algorithms since the entire Book/Member object could be found by searching the ISBN or member ID. Despite working on different parts of the code simultaneously, our planning and communication allowed for our separate parts to integrate together seamlessly and prevented setbacks from being costly.

7.2 What You Would Do Differently

Instructor Note: *If you were to redo this project, what would you change? This could include algorithm choices, data structure choices, the way you tested, how you divided work, or the overall design.*

Showing this kind of critical self-reflection is a sign of intellectual growth — do not skip this section.

If we were re-doing this project, we would have the foresight to not bother trying to include an SQL database as it ended up being convoluted and far too inefficient for the purposes of this project. We would also modify the functions to return a value instead of being void functions with print statements. The print statements work fine for this project, but returning a value would allow for better scalability and modularity in the future.

7.3 Individual Team Member Reflections

Each team member writes a short personal reflection.

Instructor Note: *Every member of the team must contribute their own 3–5 sentence reflection. Address: (a) What was your personal contribution to the project? (b) What did YOU learn that you did not know before — about algorithms, complexity, programming, or teamwork? (c) How does this project connect to your future goals or interests?*

Write your reflection in the first person (use 'I').

Team Member 1 — Ethan Winch:

I focused on creating the data structures such as the Member and Book classes as well as implementing the functions for those classes. I learned about the chronos library for timing how long it took our algorithms to run as well as importing CSV files and using the data in C++. This project has helped me to better understand the applications of the algorithms we studied in class and I've learned how to implement them in C++.

Team Member 2 — Shalin Valerio:

I focused on the creation of the test bench and algorithm and I personally learned a lot about small nuances of C++. wrapping my head around vector manipulation and writing style. BMCC did a good job of whipping me into shape but this project definitely kept me attentive to clear code to make my partner's life easier. An additional aspect was understanding how costly our class in Book.h would be as it gets manipulated through the merge and sort algorithms. It was the only reason that our searches had to be measured on the scale of nanoseconds but for the sorting it jumped up wildly into milliseconds scale.

8. Generative AI Usage Disclosure

Instructor Note: *Per the course's academic integrity policy, you MUST disclose any use of Generative AI tools (ChatGPT, Copilot, Gemini, Claude, etc.). If you did not use any AI tools, state that explicitly.*

Acceptable uses: brainstorming, clarifying concepts, getting unstuck on syntax, generating placeholder data. NOT acceptable: copying AI-generated text or code and submitting it as your own work.

For each use of AI, specify: (a) which tool, (b) what you used it for, (c) how you verified or modified the AI output.

Example: 'We used ChatGPT to brainstorm graph representation options. The AI suggested adjacency matrices and lists; we independently researched both and chose adjacency lists based on our graph's sparsity. Some ideas for the report structure were inspired by this process.'

Did your team use Generative AI tools? X Yes □ No

Tool used: ChatGPT

Purpose: Debugging why the Book and Member classes weren't able to reference each other.

How you verified or modified the output: It was a simple two line fix (including "class Member;" in Book.h and "class Book;" in Member.h). The classes were able to reference each other once the imports were made correctly.

Tool used: ChatGPT

Purpose: Debugging testbench.h.

How you verified or modified the output: a couple of instances where I missed typed a variable in the incorrect slot as the naming convention was variable_name_1xx.

9. GitHub Repository

> **Instructor Note:** *Your code must be submitted via a public GitHub repository. This section documents that repository.*
>
> *Requirements: (a) The repo must be PUBLIC by the submission deadline. (b) It must include a README.md file (see 9.2 below). (c) Commit history should reflect ongoing development — not a single last-minute upload.*

9.1 Repository Link

GitHub URL: https://github.com/ethwinch/Library-Management-System/

Appendix B: README Checklist

Confirm that your README.md includes each of the following:

> **Instructor Note:** *Your README.md is the first thing a reader sees. It must be written in clear English and include all items below. Use Markdown formatting in the README.*

- Project title and one-paragraph description of what the system does
- Names of all team members
- Programming language and any libraries/dependencies required
- Instructions: how to compile/run the program (step-by-step)
- Sample input and expected output (or link to sample files)
- Brief description of the algorithms implemented
- Link to this report (uploaded to the /docs folder)

Appendix B: Grading Rubric Summary

The table below summarizes the grading criteria from the course project description. Use it to self-check your submission before the deadline.

Component	Criterion	Details
Presentation (5%)	Slides (1 pt)	Clarity, timely, well designed, organized, easy to follow
	Communication (2 pts)	Presenter spoke clearly and effectively
	Knowledge (1 pt)	Good understanding of the topic demonstrated
	Q&A (1 pt)	Presenter responded effectively to audience questions
Report (10%)	Structure (2 pts)	Abstract, Introduction, Algorithm Architecture, Data Structure, Pseudocode, Experiments
	Writing (1 pt)	Well written, no grammar mistakes
	Algorithm Analysis (2 pts)	Computing running time correctly
	Experiments (3 pts)	Output answers the problematic questions; theory vs. practice interpretation
	Complete parts (2 pts)	Design, analysis, implementation, and testing all present
Code (10%)	Correctness (2 pts)	Program specifications and correctness
	Readability (1 pt)	Indentation, organization
	Documentation (2 pts)	Well documented; comments for each function and code section
	Efficiency (3 pts)	No errors = 3, some errors = partial, non-functional = 0
	Outputs (2 pts)	Appropriate results that solve the business problem chosen

Self-Assessment Reminder: Before submitting, re-read each row of this rubric and confirm that your report directly addresses every criterion.